



(/rol/app/)

Help

Korean

검색

Seunghe...

Sites

홈 (/rol/app/) (home) > Introducing OpenShift Service Mesh > Chapter 1: Introducing OpenShift Service Mesh

Introducing OpenShift Service Mesh

Lesson

Lab Environment

Exploring the OpenShift Service Mesh Architecture

Objectives

- Explain the high-level architecture of OpenShift Service Mesh.
- Differentiate between the control plane and data plane.
- Identify the role of OpenShift Service Mesh's core components.
- Describe how OpenShift Service Mesh integrates with Openshift.

Red Hat OpenShift Service Mesh Architecture

Red Hat OpenShift Service Mesh runs on top of Red Hat OpenShift Container Platform, and it is composed of two parts:

- OpenShift Service Mesh resources, based on the open source Istio project
- Kiali provided by Red Hat, based on the open source Kiali project.

The following diagram shows the high-level components.

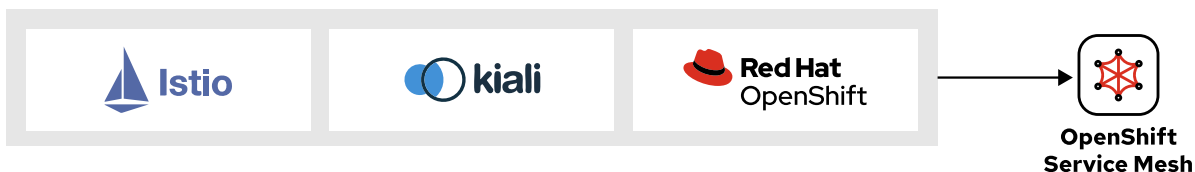


Figure 1.2: OpenShift Service Mesh High Level Components

Control Plane and Data Plane Architecture

The architecture of a service mesh is logically split into a data plane and a control plane.

Component	Description
Control Plane	The control plane is the management center of the service mesh. It does not modify any network packets in the data plane. The control plane configures the data plane to enforce policies and collect telemetry data.
Data Plane	The data plane is composed of a set of intelligent proxies. The service mesh deploys these proxies alongside the application containers and intercepts all network communication between microservices.

NOTE

Control and data plane are explained in more detail in the following section.



The Sail Operator and Istio CRDs

In OpenShift Service Mesh, the *Sail* operator is responsible for deploying the following Istio custom resources:

Istio

This cluster-wide resource deploys and manages Istio control planes.

IstioRevision

This resource represents a revision to the control plane. Having revision names enables you to use multiple independent control plane versions within the same cluster. Thus, you can install a new version of the control plane alongside an existing one. Then, you can gradually migrate your workloads to the new revision for safe, canary-style upgrades.

Each revision includes its own `istiod` deployment. The mesh workloads select a revision by using the `istio.io/rev=<revision-name>` namespace label, which configures sidecar containers to communicate with that specific revision's `istiod` instance.

NOTE

When you use a revision-based upgrade strategy, changes to the `Istio` resource automatically create corresponding control plane revisions with auto-generated names.

IstioRevisionTag

This resource provides a convenient alias for control plane revisions, simplifying canary upgrades by avoiding manual namespace relabeling.

Instead of labeling namespaces with specific revision names such as `istio.io/rev=1-2-3`, you can use a tag such as `istio.io/rev=production`. The `IstioRevisionTag` resource points to the tag of a specific revision. You can upgrade the control plane by changing the tag to point to the new revision: the control plane automatically rolls out changes to relevant workloads.

The following YAML snippet is an `IstioRevisionTag` custom resource sample:

```
apiVersion: sailoperator.io/v1
kind: IstioRevisionTag
metadata:
  name: production
  namespace: istio-system
spec:
  targetRef:
    kind: IstioRevision
    name: 1-2-3
```

1 The name of the tag. Istio labels namespaces with `istio.io/rev=production`.

2 The control plane revision that the tag points to.

IstioCNI

This resource manages the Container Network Interface (CNI) plug-in separately from the control plane, instructing the Sail operator to deploy networking pods on all cluster nodes.

Traditionally, OpenShift Service Mesh manages applications in *sidecar mode* by injecting an Envoy proxy sidecar container into each application pod and routing all communication through it. OpenShift Service Mesh also configures a privileged init container for each managed pod to set up network redirection rules before the application and Envoy containers start.

The Istio CNI plug-in enables setting up the netfilter rules without the need for an init container, but it is optional in sidecar mode. For enterprise environments with strict security requirements, choosing the CNI is the preferred option.

In *ambient mode*, sidecar-less deployment, the CNI plug-in is required.

NOTE

The details of the sidecar and ambient modes implementation are discussed below.

The OpenShift Service Mesh operator may create additional instances of the `IstioRevision` resource, depending on the update strategy defined in the `Istio` resource.

IMPORTANT

By creating custom resources such as `Istio` and `IstioCNI`, platform administrators deploy and configure the control plane. By creating custom resources such as `VirtualService` and `DestinationRule`, developers configure the communication rules for their applications.

OpenShift Service Mesh Integrations

OpenShift Service Mesh extends its capabilities through integration with other OpenShift platform components. These integrations enhance observability, security, and deployment management for service mesh workloads.

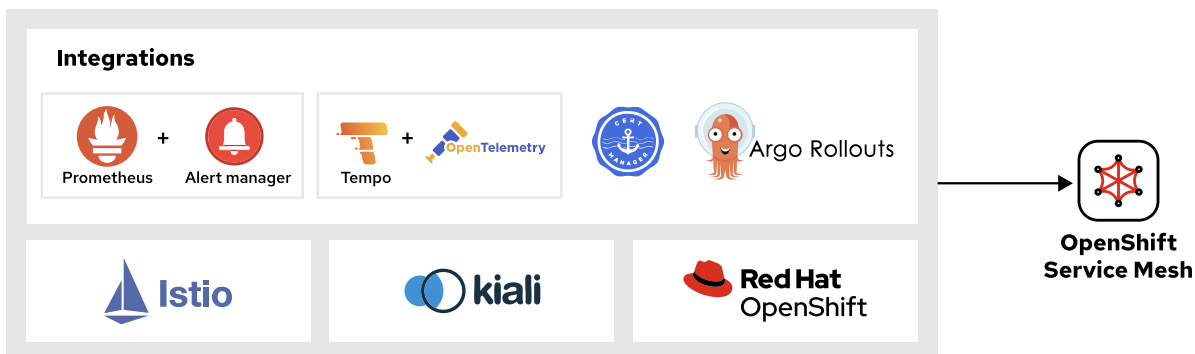


Figure 1.3: OpenShift Service Mesh with OpenShift Integrations

OpenShift Monitoring

OpenShift Service Mesh integrates with the OpenShift monitoring stack to provide centralized observability. The integration uses Prometheus to scrape, store, and query metrics from service mesh components and the applications within the mesh. The *OpenShift Service Mesh Console Plug-in (OSSMCP)* is the observability tool for Service Mesh. OSSMCP relies on these Prometheus metrics to visualize the mesh topology, display traffic flow, and report on the health of your services.

By integrating OpenShift Service Mesh with OpenShift Monitoring, you gain a centralized solution for observability. You can create custom dashboards in Grafana and configure alerts in AlertManager by using the same metrics that Kiali uses, providing a consistent view of your application and infrastructure performance.

OpenShift Distributed Tracing and OpenTelemetry Platforms

Based on Grafana Tempo and OpenTelemetry, this platform provides distributed tracing to monitor and troubleshoot transactions for complex systems. Refer to the *Observing the OpenShift Service Mesh* training content for more information about distributed tracing.

OpenShift Cert Manager

By default, OpenShift Service Mesh generates a self-signed certificate authority (CA) for ciphered communication. However, some enterprises have specific security requirements that include centralizing the use of a Public Key Infrastructure (PKI) trusted across the organization. OpenShift Cert Manager is an OpenShift operator that automates the management and issuance of TLS certificates from multiple issuing sources. Integrating OpenShift Service Mesh with Cert Manager provides several key benefits, such as the following:

- It enables the service mesh to obtain workload certificates from an enterprise's existing CA, such as Active Directory Certificate Services (ADCS) or a commercial CA. This ensures that the certificates used within the mesh are compliant with corporate security policies.
- It also centralizes certificate lifecycle management, including issuance and renewal, through a single, automated tool.

When the control plane needs a certificate for a workload proxy, it sends a certificate signing request (CSR) to the `istio-csr` agent. Then, the `istio-csr` agent creates a `CertificateRequest` Kubernetes resource. Finally, Cert Manager controllers detect this resource and fulfill the request by using a pre-configured `Issuer` or `ClusterIssuer`.

Argo Workflows

Standard Kubernetes `Deployment` resources support basic rolling updates, but complex applications need sophisticated release processes. *Argo Rollouts* provide advanced deployment capabilities, including blue-green, canary, and experiment-style releases.

The integration of Argo workflows with OpenShift Service Mesh enables fine-grained traffic management for *canary deployments*. A canary deployment involves directing a small percentage of user traffic to a new version of a service, whereas most of the traffic continues to go to the stable version. OpenShift Service Mesh uses Istio `VirtualService` resources to precisely control the distribution of traffic based on weights. Argo Rollouts can automate the manipulation of these `VirtualService` resources. This automation permits a gradual and controlled shift of traffic to the canary version. You can pause or roll back automatically the process, if metrics indicate a problem with the new version.

NOTE

Refer to the references section to learn more about each OpenShift Service Mesh integration.

OpenShift Service Mesh Control and Data Plane

OpenShift Service Mesh implements a clear separation between the control plane and data plane components. The control plane manages and configures the service mesh, and the data plane handles the actual traffic flow between services. This architectural separation provides scalability, maintainability, and operational flexibility.

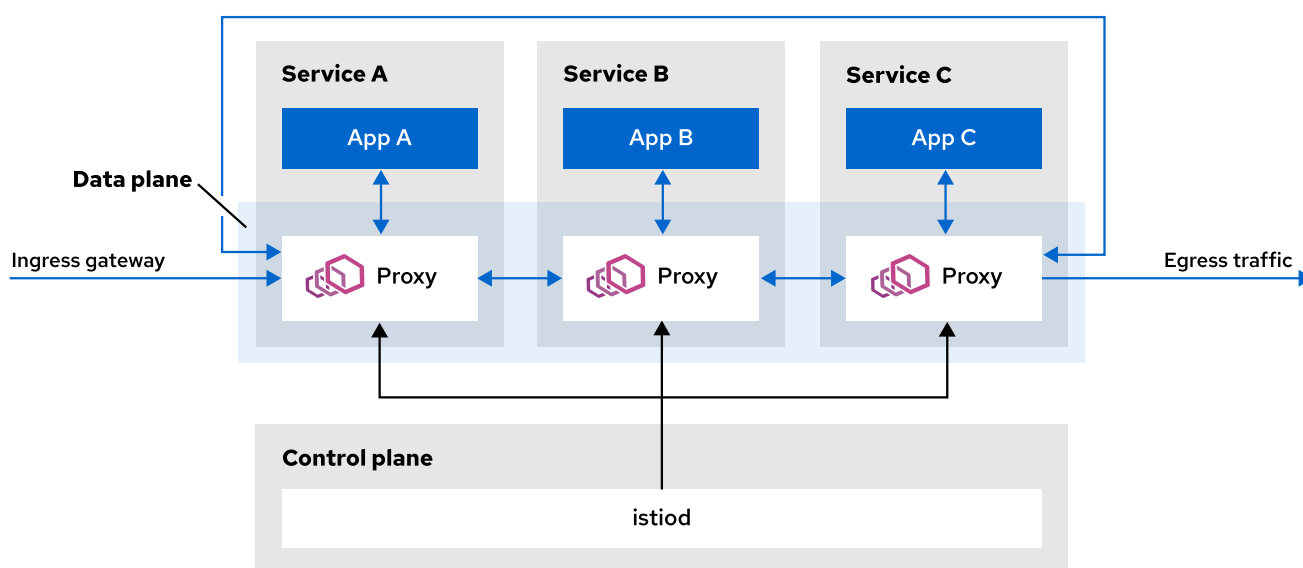


Figure 1.4: Control Plane and Data Plane Architecture

OpenShift Service Mesh Control Plane: `istiod`

The OpenShift Service Mesh control plane provides the management and configuration for the service mesh. It acts as the authority for the data plane proxies.

In OpenShift Service Mesh 3, the control plane is consolidated into a single `istiod` binary, replacing the multiple separated components in older versions. This simplifies installation, management, and operation.

The `istiod` binary includes the following core functions:

Service discovery

The `istiod` tracks services and endpoints, providing discovery information to Envoy proxies. This enables a service to find and communicate with other services without needing to know their specific IP addresses.

Configuration management and propagation

An application developer defines high-level traffic routing rules by using Red Hat OpenShift and Istio resources, such as `VirtualService` and `DestinationRule`. The `istiod` translates these high-level rules into Envoy-specific configurations. Then, `istiod` distributes these configurations to the Envoy sidecar proxies across the mesh.

Certificate authority (CA)

The `istiod` includes a built-in CA. It generates and distributes TLS certificates to the Envoy proxies in the data plane. These certificates secure service-to-service communication by using mutual TLS (mTLS) authentication and encryption.

OpenShift Service Mesh Data Plane: Envoy Proxy

The data plane handles traffic between services within the mesh, ensuring reliable and secure communication based on the configuration of the control plane.

OpenShift Service Mesh uses an extended version of the Envoy proxy, deployed as a sidecar container alongside each application. This high-performance, language-agnostic proxy intercepts all network traffic transparently, providing:

- Dynamic service discovery and load balancing
- TLS termination and HTTP/2/gRPC proxying
- Circuit breakers, health checks, and fault injection
- Staged rollouts and rich metrics collection

The out-of-process architecture decouples network logic from application logic, enabling consistent policies across diverse services.

The data plane at the network transport layer

Envoy operates at Layer 4, the *transport layer*, for TCP/UDP traffic, routing based on IP addresses and ports. Envoy includes a pluggable filter chain mechanism that gives different TCP proxy tasks. You can insert filters to support multiple tasks, such as raw TCP proxying and client certificate authentication for TLS.

NOTE

Layer 4 proxying is also suitable for non-HTTP services such as databases or message queues.

The data plane at the network application layer

At Layer 7, the *application layer*, Envoy provides extensive HTTP support, understanding and manipulating request content, including headers, paths, and cookies. This enables sophisticated traffic management with fine-grained routing rules based on HTTP methods, headers, or query parameters, crucial for canary releases and A/B testing.

Control Plane and Data Plane Interaction

The control plane does not directly participate in the runtime traffic between services. Instead, it programs the data plane. The interaction works as follows:

1. The developer creates or updates OpenShift Service Mesh configuration resources, such as a `VirtualService`.
2. `istiod` watches for changes to these resources.
3. `istiod` converts the high-level rule to Envoy-specific configurations.
4. `istiod` securely distributes this new configuration to relevant Envoy sidecar proxies.
5. Envoy proxies apply the new configuration dynamically without service downtime.

This process lets administrators and developers manage traffic routing, enforce security policies, and monitor service behavior centrally and in real-time. The Envoy proxies continuously send telemetry data, such as metrics and traces, to monitoring systems, which provides observability into the mesh.

Extending Data Plane with Web Assembly

You can customize the Envoy's extensible filter-based architecture through WebAssembly (WASM) plug-ins. WASM is a portable binary format that enables custom logic without modifying the core proxy.

WASM plug-ins provide key advantages:

Efficiency

WASM extensions add low latency, CPU, and memory overhead because they are compiled to a compact binary format that runs in a lightweight, sandboxed environment.

Isolation and Security

A programming error or crash in one WASM plug-in does not affect other plug-ins or the Envoy proxy. Each plug-in runs in a secure sandbox with limited access to system resources.

Flexibility

Support for multiple programming languages (C++, Rust, Go).

Dynamic Configuration

Load, update, and unload plug-ins without restarting Envoy.

The `WasmPlugin` custom resource is used to configure a WASM extension for a workload or proxy.

By using WASM, platform teams can develop and deploy custom logic to handle specific use cases, such as custom authentication schemes, proprietary telemetry formats, or complex traffic transformations, without modifying the core Envoy proxy.

Kiali

Kiali is the primary observability and management console for OpenShift Service Mesh. It provides a comprehensive graphical interface for visualizing, monitoring, and managing your service mesh.

Refer to the *Observing OpenShift Service Mesh* training content for more information about Kiali and OSSMCP.

Data Plane Deployment Models

OpenShift Service Mesh offers two distinct approaches for deploying the data plane: the traditional *sidecar* model and the newer *sidecar-less ambient* model. Each approach has different characteristics, benefits, and trade-offs that make it suitable for different use cases and requirements.

The choice between these deployment models affects resource consumption, security posture, operational complexity, and performance characteristics of your service mesh. Understanding both models helps you make informed decisions about which approach best suits your specific needs.

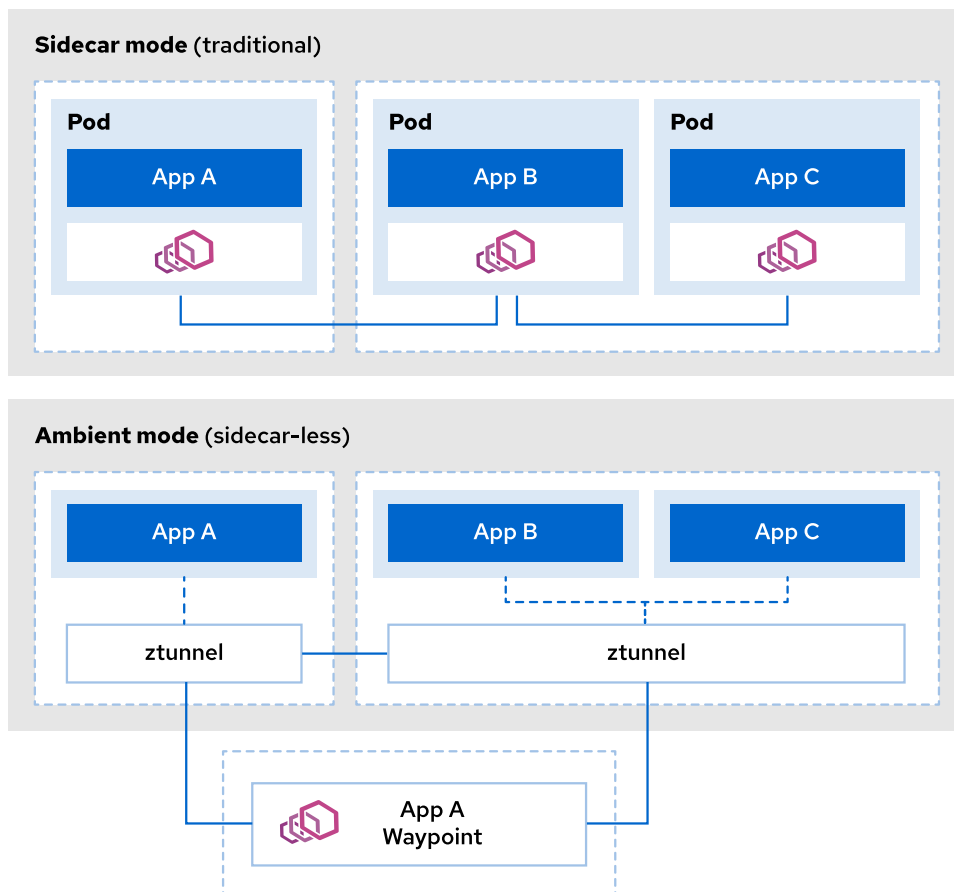


Figure 1.5: Data Plane Deployment Models Comparison

In the sidecar deployment model, an Envoy proxy is injected into each application's pod. The proxy intercepts all inbound and outbound traffic. This interception is transparent to the application container. These characteristics provide several benefits already discussed above, but they also introduce some challenges, as follows:

- Resource overhead per pod
- Scaling challenges
- Performance impact

Ambient Mode

To address the challenges of the sidecar model, especially the resource overhead, a new data plane mode named ambient mesh is available. Ambient mesh takes a sidecar-less approach.

NOTE

In OpenShift Service Mesh 3.1, Istio ambient mode is available as a Technology Preview feature. This status indicates that it is suitable for development and pre-production environments. The generally available and fully supported data plane mode remains the sidecar model.

The ambient mode splits the functionality of the service mesh into two distinct layers that run on each node in the cluster.

The first layer is a secure overlay that handles Layer 4 (L4) telemetry, authentication, and encryption. A node-level agent named `ztunnel` provides this functionality. The `ztunnel` establishes mTLS connections and enforces L4 authorization policies. Because `ztunnel` runs one instance per node instead of one per pod, it significantly reduces resource overhead compared to the sidecar model. All pods on a node share the same `ztunnel` for secure communication.

The second layer provides Layer 7 (L7) service mesh features, such as traffic routing, load balancing, and L7 authorization policies. Envoy-based *waypoint proxies* handle this functionality. You deploy a *waypoint proxy* per namespace or service account as needed. Traffic is routed from the `ztunnel` to a *waypoint proxy* only when a service in the destination namespace requires L7 processing.

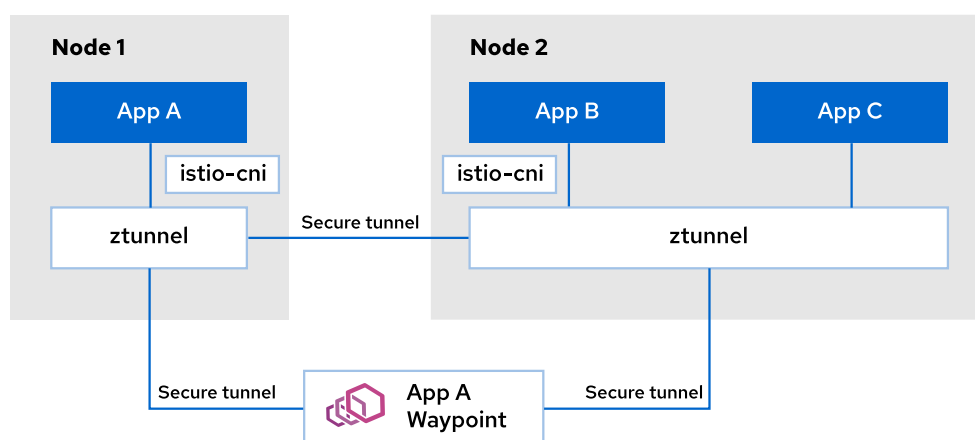


Figure 1.6: Ambient Mode Components

This layered approach ensures that resources for L7 processing are only consumed when necessary, which further optimizes performance and latency. If a workload only requires mTLS and L4 policies, then its traffic goes through only the `ztunnel`. If it requires L7 policies, then its traffic goes through both the `ztunnel` and the *waypoint proxy*.

NOTE

This course is focused on the sidecar mode. Refer to the references section for more information about ambient mode.

Scaling OpenShift Service Mesh

As organizations grow and their service mesh deployments expand, they require effective strategies to scale their service mesh infrastructure. OpenShift Service Mesh provides several approaches to handle increased workloads, multiple teams, and distributed environments while maintaining security, isolation, and operational efficiency.

This section introduces the key scaling strategies available in OpenShift Service Mesh, including multitenancy for team isolation, multicluster deployments for geographic distribution, and the architectural considerations that enable these scaling approaches.

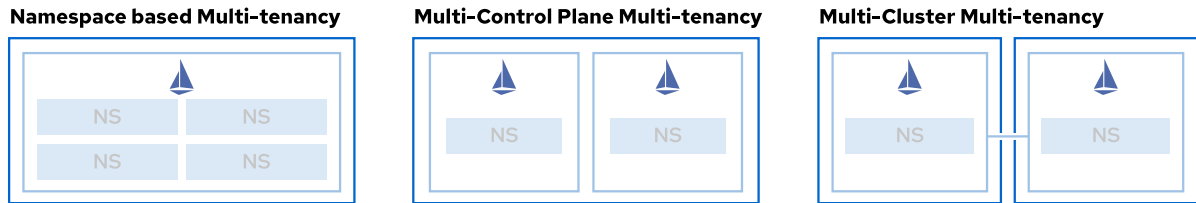


Figure 1.7: Scaling OpenShift Service Mesh

Multitenancy Features

OpenShift Service Mesh provides multitenancy capabilities to isolate service mesh deployments within a single cluster. This enables different teams or applications to have their own service mesh without interfering with each other. The primary mechanism for achieving multitenancy is through namespace-based isolation.

By default, OpenShift Service Mesh uses a single control plane. You can specify which namespaces are part of the service mesh by using label selectors and labeling namespaces accordingly. This ensures that the control plane only manages the resources within its designated namespaces. The Istio resource lets you define which discovery selectors a particular control plane instance uses.

For stronger isolation, you can deploy multiple independent service mesh control planes in a single cluster. Each control plane manages its own set of namespaces, effectively creating separate service meshes. This is useful in scenarios where different teams require different service mesh configurations or versions.

To deploy multiple control planes, you use a combination of `discoverySelectors` and `revisions`. The `discoverySelectors` field in the Istio resource specifies which namespaces the control plane should watch. The `revision` field assigns a unique name to the control plane, which is used to associate namespaces with that control plane.

If the isolation requirements include being able to communicate only to workloads in the same service mesh, then you must use `PeerAuthentication` resources in each control plane namespace to enforce strict mTLS across the mesh.

Multicluster Deployments

OpenShift Service Mesh supports extending a single service mesh across multiple clusters. This provides high availability and fault tolerance for the service mesh features your applications depend upon.

There are three primary models for multicluster control plane deployments: multi-primary, primary-remote, and external control plane.

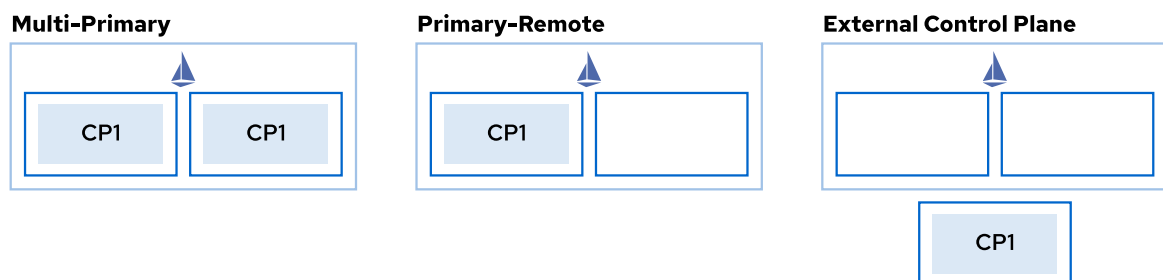


Figure 1.8: Multicluster Control Plane Deployment Models

In a multi-primary deployment, each cluster has its own control plane. Control planes discover services in all clusters, providing a unified view of the service mesh. This model offers high availability for the control plane.

In a primary-remote deployment, one cluster hosts the control plane, and other clusters connect to it as remotes. This centralizes the management of the service mesh. However, the control plane becomes a single point of failure.

In an external control plane deployment, one cluster hosts and manages the Istio control plane, while applications are deployed on other clusters. This topology improves security and enables the Service Mesh to be hosted as a service.

In addition to control plane topologies, a multicluster mesh must also use one of the two available network topologies: single or multinetwork.

In a single network topology, all clusters reside on the same network, and there is direct connectivity between services within all clusters. There is no need to use gateways for communication between the services across cluster boundaries.

In a multinet topology, clusters reside on different networks, and there is no direct connectivity between services. Gateways must be used to enable communication across network boundaries.

This control plane and network model separation obsoletes the *federated mesh* topology from older versions.

NOTE

Refer to the references section to learn more about multitenancy and multicluster deployments.

REFERENCES

For more information about OpenShift Service Mesh 3.1 architecture and features, refer to the *About OpenShift Service Mesh* documentation at https://docs.redhat.com/en/documentation/red_hat_openshift_service_mesh/3.1/html-single/about/index (https://docs.redhat.com/en/documentation/red_hat_openshift_service_mesh/3.1/html-single/about/index)

For more information about deploying and managing the control plane, refer to the *Installing OpenShift Service Mesh* documentation at https://docs.redhat.com/en/documentation/red_hat_openshift_service_mesh/3.1/html-single/installing/index (https://docs.redhat.com/en/documentation/red_hat_openshift_service_mesh/3.1/html-single/installing/index)

For more information about sidecar injection, refer to the *Sidecar Injection* chapter of *Installing OpenShift Service Mesh* documentation at https://docs.redhat.com/en/documentation/red_hat_openshift_service_mesh/3.1/html-single/installing/index#ossm-sidecar-injection (https://docs.redhat.com/en/documentation/red_hat_openshift_service_mesh/3.1/html-single/installing/index#ossm-sidecar-injection)

For more information about configuring ambient mesh, refer to the *Istio ambient mode* chapter of *Installing OpenShift Service Mesh* documentation at https://docs.redhat.com/en/documentation/red_hat_openshift_service_mesh/3.1/html-single/installing/index#ossm-istio-ambient-mode (https://docs.redhat.com/en/documentation/red_hat_openshift_service_mesh/3.1/html-single/installing/index#ossm-istio-ambient-mode)

For more information about multicluster topologies, refer to the *Multicluster Topologies* chapter of *Installing OpenShift Service Mesh* documentation at https://docs.redhat.com/en/documentation/red_hat_openshift_service_mesh/3.1/html-single/installing/index#ossm-multi-cluster-topologies (https://docs.redhat.com/en/documentation/red_hat_openshift_service_mesh/3.1/html-single/installing/index#ossm-multi-cluster-topologies)

For more information about OpenShift Monitoring, refer to the *Monitoring* documentation at https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/monitoring/index (https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/monitoring/index)

For more information about Red Hat build of OpenTelemetry, refer to the *Red Hat build of OpenTelemetry* documentation at https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/red_hat_build_of_opentelemetry/index (https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/red_hat_build_of_opentelemetry/index)

For more information about Distributed Tracing, refer to the *Distributed Tracing* documentation at https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/distributed_tracing/index (https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/distributed_tracing/index)

For more information about cert-manager operator, refer to the *cert-manager Operator for Red Hat OpenShift* chapter of *Security and compliance* documentation at https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/security_and_compliance/index#cert-manager-operator-for-red-hat-openshift (https://docs.redhat.com/en/documentation/openshift_container_platform/4.18/html-single/security_and_compliance/index#cert-manager-operator-for-red-hat-openshift)

For more information about Argo Rollouts, refer to the *Argo Rollouts* documentation at https://docs.redhat.com/en/documentation/red_hat_openshift_gitops/1.17/html-single/argo_rollouts/index (https://docs.redhat.com/en/documentation/red_hat_openshift_gitops/1.17/html-single/argo_rollouts/index)

Introducing OpenShift Service Mesh 3.1 (<https://www.redhat.com/en/blog/introducing-openshift-service-mesh-31>)

Try Istio ambient mode on Red Hat OpenShift (<https://developers.redhat.com/articles/2025/03/12/try-istio-ambient-mode-red-hat-openshift#>)

Istio.io: Istio Architecture (<https://istio.io/v1.26/docs/ops/deployment/architecture/>)



개인 정보 취급 방침 (https://www.redhat.com/ko/about/privacy-policy?extIdCarryOver=true&sc_cid=701f20000001D8QoAAK)

Red Hat 교육 정책 (<https://www.redhat.com/ko/about/red-hat-training-policies>)

이용 약관 (<https://www.redhat.com/ko/about/terms-use>)

모든 정책 및 지침 (<https://www.redhat.com/ko/about/all-policies-guidelines>)

릴리스 노트
(https://docs.redhat.com/en/documentation/red_hat_learning_subscription/1-latest/html-single/red_hat_learning_subscription_release_notes/index)

쿠키 기본설정